

aux_meses.sql

```
-- =====
-- PASSO 1: Cria tabela física de meses (não temporária)
-- =====
DROP TABLE IF EXISTS aux_meses;

CREATE TABLE aux_meses AS
SELECT
    sk_tempo,
    data_completa
    LAST_DAY(data_completa) AS primeiro_dia,
    AS ultimo_dia,
    DATE_FORMAT(data_completa, '%Y-%m') AS ano_mes,
    YEAR(data_completa) AS ano,
    MONTH(data_completa) AS mes
FROM dim_tempo
WHERE dia_mes = 1
ORDER BY data_completa;

-- Verifica: deve retornar 264
SELECT COUNT(*) AS total_meses FROM aux_meses;
```

auxiliares_headcount.sql

```
-- =====
-- PASSO 2: Tabela auxiliar de admissões por mês/loja/cargo
-- =====
DROP TABLE IF EXISTS aux_admissoes;

CREATE TABLE aux_admissoes AS
SELECT
    DATE_FORMAT(data_admissao, '%Y-%m') AS ano_mes,
    sk_loja,
    sk_cargo,
    COUNT(*) AS total_admissoes
FROM dim_colaborador
WHERE data_admissao BETWEEN '2005-01-01' AND '2026-12-31'
GROUP BY
    DATE_FORMAT(data_admissao, '%Y-%m'),
    sk_loja,
    sk_cargo;

-- Verifica
SELECT COUNT(*) AS total FROM aux_admissoes;

-- =====
-- PASSO 3: Tabela auxiliar de demissões por mês/loja/cargo
-- =====
DROP TABLE IF EXISTS aux_demissoes;

CREATE TABLE aux_demissoes AS
SELECT
    DATE_FORMAT(data_demissao, '%Y-%m') AS ano_mes,
    sk_loja,
    sk_cargo,
    COUNT(*) AS total_demissoes
FROM dim_colaborador
WHERE data_demissao IS NOT NULL
GROUP BY
    DATE_FORMAT(data_demissao, '%Y-%m'),
    sk_loja,
    sk_cargo;

-- Verifica
SELECT COUNT(*) AS total FROM aux_demissoes;
```

```

-- =====
-- PASSO 4: Tabela auxiliar de headcount orçado
-- =====
DROP TABLE IF EXISTS aux_orcado;

CREATE TABLE aux_orcado AS
SELECT
    DATE_FORMAT(
        STR_TO_DATE(CONCAT(ho.ano, '-', LPAD(ho.mes, 2, '0'), '-01'), '%Y-%m-%d'),
        '%Y-%m'
    ) AS ano_mes,
    dl.sk_loja,
    dc.sk_cargo,
    SUM(ho.total_headcount_orcado) AS headcount_orcado
FROM headcount_orcado ho
INNER JOIN dim_loja dl ON dl.nome_loja = ho.nome_loja
INNER JOIN dim_cargo dc ON dc.departamento = ho.departamento
                        AND dc.centro_custo = ho.centro_custo
                        AND dc.funcao = ho.funcao

GROUP BY
    ano_mes,
    dl.sk_loja,
    dc.sk_cargo;

-- Verifica
SELECT COUNT(*) AS total FROM aux_orcado;

```

dim_cargo.sql

```

-- =====
-- DIM_CARGO
-- Consolida departamento, centro de custo, função e nível
-- =====

DROP TABLE IF EXISTS dim_cargo;

CREATE TABLE dim_cargo (
    sk_cargo          INT          NOT NULL AUTO_INCREMENT,
    departamento      VARCHAR(20)  NOT NULL,
    centro_custo      VARCHAR(40)  NOT NULL,
    funcao            VARCHAR(60)  NOT NULL,
    nivel_cargo       VARCHAR(15)  NOT NULL, -- 'Operacional', 'Analista', 'Gestão'
    familia_cargo     VARCHAR(30)  NOT NULL, -- 'RH', 'Financeiro', 'TI', etc.
    PRIMARY KEY (sk_cargo),
    UNIQUE KEY uq_cargo (departamento, centro_custo, funcao)
);

-- =====
-- Popular dim_cargo a partir da base de colaboradores
-- =====
INSERT INTO dim_cargo (departamento, centro_custo, funcao, nivel_cargo, familia_cargo)
SELECT DISTINCT
    departamento,
    centro_custo,
    funcao,
    -- Nível do cargo
    CASE
        WHEN funcao LIKE '%Diretor%'
        OR funcao LIKE '%Gerente%'
        OR funcao LIKE '%Coordenador%' THEN 'Gestão'
        WHEN funcao LIKE '%Analista%'
        OR funcao LIKE '%Engenheiro%'
        OR funcao LIKE '%Advogado%'
        OR funcao LIKE '%Arquiteto%'
        OR funcao LIKE '%Cientista%'
        OR funcao LIKE '%Controller%'
        OR funcao LIKE '%Auditor%'

```

```

        OR funcao LIKE '%Médico%'
        OR funcao LIKE '%Enfermeiro%'
        OR funcao LIKE '%Especialista%'
        OR funcao LIKE '%Business Partner%'
        OR funcao LIKE '%Scrum Master%'
        OR funcao LIKE '%Product Owner%'
        OR funcao LIKE '%PMO%'
        OR funcao LIKE '%Tech Lead%'
        OR funcao LIKE '%Tech Recruiter%'
        OR funcao LIKE '%Comprador%'
        OR funcao LIKE '%Tesoureiro%'
        OR funcao LIKE '%Desenvolvedor%' THEN 'Analista'
    ELSE
        'Operacional'
END,
-- Família do cargo
CASE
    WHEN centro_custo IN ('Departamento Pessoal','DHO',
        'Recrutamento e Seleção',
        'RH - CD','RH - Lojas') THEN 'Recursos Humanos'
    WHEN centro_custo IN ('Financeiro','Controladoria') THEN 'Financeiro'
    WHEN centro_custo IN ('Jurídico') THEN 'Jurídico'
    WHEN centro_custo IN ('Tecnologia','Suporte e Service Desk',
        'Infraestrutura','Desenvolvimento',
        'Inteligência de Negócios') THEN 'Tecnologia'
    WHEN centro_custo IN ('Logística','Depósito',
        'Produção','Manutenção') THEN 'Operações'
    WHEN centro_custo IN ('Vendas','Caixas','Estoque') THEN 'Comercial'
    WHEN centro_custo IN ('SESMT','SESMT - CD',
        'SESMT - Lojas') THEN 'Saúde e Segurança'
    WHEN centro_custo IN ('Prevenção de Perdas') THEN 'Prevenção de Perdas'
    WHEN centro_custo IN ('Compras') THEN 'Compras'
    WHEN centro_custo IN ('Planejamento') THEN 'Planejamento'
    WHEN centro_custo IN ('Projeto Inovação') THEN 'Inovação'
    WHEN centro_custo IN ('Frotas') THEN 'Frotas'
    WHEN centro_custo IN ('Facilities','Limpeza') THEN 'Facilities'
    WHEN centro_custo IN ('Qualidade') THEN 'Qualidade'
    WHEN centro_custo IN ('Diretoria') THEN 'Diretoria'
    ELSE
        'Outros'
END
FROM colaboradores
ORDER BY departamento, centro_custo, funcao;

-- =====
-- Verificar
-- =====
SELECT
    nivel_cargo,
    COUNT(*) AS total_cargos
FROM dim_cargo
GROUP BY nivel_cargo
ORDER BY nivel_cargo;

```

dim_colaborador.sql

```

-- =====
-- DIM_COLABORADOR
-- Dimensão principal com todos os atributos do colaborador
-- =====

DROP TABLE IF EXISTS dim_colaborador;

CREATE TABLE dim_colaborador (
    sk_colaborador          INT          NOT NULL AUTO_INCREMENT,
    matricula_colaborador   VARCHAR(8)   NOT NULL,
    nome_colaborador        VARCHAR(100)  NOT NULL,
    sexo                   VARCHAR(10)    NOT NULL,
    data_nascimento         DATE          NOT NULL,

```

```

faixa_etaria          VARCHAR(20) NOT NULL, -- '18-25', '26-35', etc.
naturalidade          VARCHAR(100) NOT NULL,
cidade_moradia        VARCHAR(50) NOT NULL,
estado_moradia        VARCHAR(2) NOT NULL,
pais                  VARCHAR(20) NOT NULL,
escolaridade          VARCHAR(40) NOT NULL,
tipo_contratacao      VARCHAR(10) NOT NULL,
situacao              VARCHAR(20) NOT NULL,
data_admissao         DATE NOT NULL,
data_demissao         DATE NULL,
tempo_casa_anos       DECIMAL(5,2) NULL, -- anos de empresa até saída ou hoje
motivo_desligamento  VARCHAR(60) NULL,
tipo_desligamento    VARCHAR(40) NULL,
sk_loja               INT NOT NULL,
sk_cargo              INT NOT NULL,
salario_contratacao   DECIMAL(10,2) NOT NULL,
vale_alimentacao      DECIMAL(8,2) NOT NULL,
auxilio_combustivel   DECIMAL(8,2) NOT NULL,
auxilio_creche        DECIMAL(8,2) NOT NULL,
PRIMARY KEY (sk_colaborador),
UNIQUE KEY uq_matricula (matricula_colaborador),
KEY fk_dc_loja (sk_loja),
KEY fk_dc_cargo (sk_cargo)
);

-- =====
-- Popular dim_colaborador
-- =====
INSERT INTO dim_colaborador (
    matricula_colaborador,
    nome_colaborador,
    sexo,
    data_nascimento,
    faixa_etaria,
    naturalidade,
    cidade_moradia,
    estado_moradia,
    pais,
    escolaridade,
    tipo_contratacao,
    situacao,
    data_admissao,
    data_demissao,
    tempo_casa_anos,
    motivo_desligamento,
    tipo_desligamento,
    sk_loja,
    sk_cargo,
    salario_contratacao,
    vale_alimentacao,
    auxilio_combustivel,
    auxilio_creche
)
SELECT
    c.matricula_colaborador,
    c.nome_colaborador,
    c.sexo,
    c.data_nascimento,
    -- Faixa etária calculada com base na idade atual
    CASE
        WHEN TIMESTAMPDIFF(YEAR, c.data_nascimento, CURDATE()) BETWEEN 17 AND 25 THEN '17-25'
        WHEN TIMESTAMPDIFF(YEAR, c.data_nascimento, CURDATE()) BETWEEN 26 AND 35 THEN '26-35'
        WHEN TIMESTAMPDIFF(YEAR, c.data_nascimento, CURDATE()) BETWEEN 36 AND 45 THEN '36-45'
        WHEN TIMESTAMPDIFF(YEAR, c.data_nascimento, CURDATE()) BETWEEN 46 AND 55 THEN '46-55'
        WHEN TIMESTAMPDIFF(YEAR, c.data_nascimento, CURDATE()) BETWEEN 56 AND 65 THEN '56-65'
        ELSE '66+'
    END,
    c.naturalidade,
    c.cidade_moradia,
    c.estado_moradia,
    c.pais,
    c.escolaridade,

```

```

c.tipo_contratacao,
c.situacao,
c.data_admissao,
c.data_demissao,
-- Tempo de casa em anos (até demissão ou até hoje)
ROUND(
    TIMESTAMPDIFF(DAY,
        c.data_admissao,
        COALESCE(c.data_demissao, CURDATE()))
    ) / 365.25
, 2),
c.motivo_desligamento,
c.tipo_desligamento,
dl.sk_loja,
dc.sk_cargo,
c.salario_contratacao,
c.vale_alimentacao,
c.auxilio_combustivel,
c.auxilio_creche
FROM colaboradores c
-- Join com dim_loja pelo nome da loja
INNER JOIN dim_loja dl ON dl.nome_loja = c.loja
-- Join com dim_cargo pela combinação departamento + centro_custo + funcao
INNER JOIN dim_cargo dc ON dc.departamento = c.departamento
                        AND dc.centro_custo = c.centro_custo
                        AND dc.funcao = c.funcao;

-- =====
-- Verificar (deve retornar 50000)
-- =====
SELECT
    situacao,
    tipo_contratacao,
    COUNT(*) AS total,
    ROUND(AVG(tempo_casa_anos), 2) AS media_anos_casa
FROM dim_colaborador
GROUP BY situacao, tipo_contratacao
ORDER BY situacao, tipo_contratacao;

```

dim_loja.sql

```

-- =====
-- DIM_LOJA
-- =====

DROP TABLE IF EXISTS dim_loja;

CREATE TABLE dim_loja (
    sk_loja          INT          NOT NULL AUTO_INCREMENT,
    nome_loja        VARCHAR(60) NOT NULL,
    tipo_loja        VARCHAR(30) NOT NULL, -- 'Matriz', 'Loja', 'Centro Distribuição'
    cidade           VARCHAR(50) NOT NULL,
    estado           VARCHAR(2)  NOT NULL,
    regioao          VARCHAR(15) NOT NULL, -- 'Sudeste', 'Nordeste', etc.
    pais             VARCHAR(20) NOT NULL,
    PRIMARY KEY (sk_loja),
    UNIQUE KEY uq_nome_loja (nome_loja)
);

-- =====
-- Popular dim_loja
-- =====
INSERT INTO dim_loja (nome_loja, tipo_loja, cidade, estado, regioao, pais)
VALUES
    ('Matriz', 'Matriz', 'São Paulo', 'SP', 'Sudeste',
     'Centro Distribuição - Campinas', 'Centro Distribuição', 'Campinas', 'SP', 'Sudeste',
     'Centro Distribuição - Maringá', 'Centro Distribuição', 'Maringá', 'PR', 'Sul',

```

```

('Centro Distribuição - Santos', 'Centro Distribuição', 'Santos', 'SP', 'Sudeste',
('Loja Anápolis', 'Loja', 'Anápolis', 'GO', 'Centro-Oeste',
('Loja Aracaju', 'Loja', 'Aracaju', 'SE', 'Nordeste',
('Loja Belém', 'Loja', 'Belém', 'PA', 'Norte',
('Loja Belo Horizonte', 'Loja', 'Belo Horizonte', 'MG', 'Sudeste',
('Loja Boa Vista', 'Loja', 'Boa Vista', 'RR', 'Norte',
('Loja Brasília', 'Loja', 'Brasília', 'DF', 'Centro-Oeste',
('Loja Campinas', 'Loja', 'Campinas', 'SP', 'Sudeste',
('Loja Campo Grande', 'Loja', 'Campo Grande', 'MS', 'Centro-Oeste',
('Loja Caruaru', 'Loja', 'Caruaru', 'PE', 'Nordeste',
('Loja Caxias do Sul', 'Loja', 'Caxias do Sul', 'RS', 'Sul',
('Loja Cuiabá', 'Loja', 'Cuiabá', 'MT', 'Centro-Oeste',
('Loja Curitiba', 'Loja', 'Curitiba', 'PR', 'Sul',
('Loja Feira de Santana', 'Loja', 'Feira de Santana', 'BA', 'Nordeste',
('Loja Florianópolis', 'Loja', 'Florianópolis', 'SC', 'Sul',
('Loja Fortaleza', 'Loja', 'Fortaleza', 'CE', 'Nordeste',
('Loja Goiânia', 'Loja', 'Goiânia', 'GO', 'Centro-Oeste',
('Loja João Pessoa', 'Loja', 'João Pessoa', 'PB', 'Nordeste',
('Loja Joinville', 'Loja', 'Joinville', 'SC', 'Sul',
('Loja Juazeiro do Norte', 'Loja', 'Juazeiro do Norte', 'CE', 'Nordeste',
('Loja Londrina', 'Loja', 'Londrina', 'PR', 'Sul',
('Loja Macapá', 'Loja', 'Macapá', 'AP', 'Norte',
('Loja Maceió', 'Loja', 'Maceió', 'AL', 'Nordeste',
('Loja Manaus', 'Loja', 'Manaus', 'AM', 'Norte',
('Loja Maringá', 'Loja', 'Maringá', 'PR', 'Sul',
('Loja Natal', 'Loja', 'Natal', 'RN', 'Nordeste',
('Loja Niterói', 'Loja', 'Niterói', 'RJ', 'Sudeste',
('Loja Palmas', 'Loja', 'Palmas', 'TO', 'Norte',
('Loja Porto Alegre', 'Loja', 'Porto Alegre', 'RS', 'Sul',
('Loja Porto Velho', 'Loja', 'Porto Velho', 'RO', 'Norte',
('Loja Recife', 'Loja', 'Recife', 'PE', 'Nordeste',
('Loja Ribeirão Preto', 'Loja', 'Ribeirão Preto', 'SP', 'Sudeste',
('Loja Rio Branco', 'Loja', 'Rio Branco', 'AC', 'Norte',
('Loja Rio de Janeiro', 'Loja', 'Rio de Janeiro', 'RJ', 'Sudeste',
('Loja Salvador', 'Loja', 'Salvador', 'BA', 'Nordeste',
('Loja Santos', 'Loja', 'Santos', 'SP', 'Sudeste',
('Loja São Luís', 'Loja', 'São Luís', 'MA', 'Nordeste',
('Loja São Paulo', 'Loja', 'São Paulo', 'SP', 'Sudeste',
('Loja Sorocaba', 'Loja', 'Sorocaba', 'SP', 'Sudeste',
('Loja Teresina', 'Loja', 'Teresina', 'PI', 'Nordeste',
('Loja Uberlândia', 'Loja', 'Uberlândia', 'MG', 'Sudeste',

-- =====
-- Verificar (deve retornar 44)
-- =====
SELECT COUNT(*) AS total_lojas FROM dim_loja;

```

dim_tempo.sql

```

-- =====
-- PASSO 1: Aumentar limite de recursão para a sessão
-- =====
SET SESSION cte_max_recursion_depth = 10000;

-- =====
-- PASSO 2: Recriar a tabela (caso já exista vazia)
-- =====
DROP TABLE IF EXISTS dim_tempo;

CREATE TABLE dim_tempo (
    sk_tempo          INT          NOT NULL AUTO_INCREMENT,
    data_completa     DATE         NOT NULL,
    ano               SMALLINT    NOT NULL,
    semestre          TINYINT     NOT NULL,
    trimestre         TINYINT     NOT NULL,
    mes               TINYINT     NOT NULL,
    nome_mes          VARCHAR(15) NOT NULL,

```

```

        semana_ano      TINYINT      NOT NULL,
        dia_mes         TINYINT      NOT NULL,
        dia_semana      TINYINT      NOT NULL,
        nome_dia        VARCHAR(15)  NOT NULL,
        dia_util        TINYINT(1)    NOT NULL,
        ano_mes         VARCHAR(7)    NOT NULL,
        ano_trimestre   VARCHAR(7)    NOT NULL,
        PRIMARY KEY (sk_tempo),
        UNIQUE KEY uq_data (data_completa)
    );

-- =====
-- PASSO 3: Popular com WITH RECURSIVE (limite já ajustado)
-- =====
INSERT INTO dim_tempo (
    data_completa, ano, semestre, trimestre, mes, nome_mes,
    semana_ano, dia_mes, dia_semana, nome_dia, dia_util,
    ano_mes, ano_trimestre
)
WITH RECURSIVE datas AS (
    SELECT DATE('2005-01-01') AS dt
    UNION ALL
    SELECT DATE_ADD(dt, INTERVAL 1 DAY)
    FROM datas
    WHERE dt < '2026-12-31'
)
SELECT
    dt,
    YEAR(dt),
    IF(MONTH(dt) <= 6, 1, 2),
    QUARTER(dt),
    MONTH(dt),
    ELT(MONTH(dt),
        'Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho',
        'Julho', 'Agosto', 'Setembro', 'Outubro', 'Novembro', 'Dezembro'),
    WEEK(dt, 3),
    DAY(dt),
    DAYOFWEEK(dt),
    ELT(DAYOFWEEK(dt),
        'Domingo', 'Segunda', 'Terça', 'Quarta', 'Quinta', 'Sexta', 'Sábado'),
    IF(DAYOFWEEK(dt) IN (1, 7), 0, 1),
    DATE_FORMAT(dt, '%Y-%m'),
    CONCAT(YEAR(dt), '-Q', QUARTER(dt))
FROM datas;

-- =====
-- PASSO 4: Verificar resultado (deve retornar 8035)
-- =====
SELECT COUNT(*) AS total_dias FROM dim_tempo;

```

fato_absenteismo.sql

```

-- =====
-- FATO_ABSENTEISMO
-- Granularidade: 1 linha por ocorrência de ausência
-- =====

DROP TABLE IF EXISTS fato_absenteismo;

CREATE TABLE fato_absenteismo (
    sk_absenteismo      INT          NOT NULL AUTO_INCREMENT,
    sk_tempo_inicio     INT          NOT NULL,
    sk_tempo_fim        INT          NOT NULL,
    sk_colaborador      INT          NOT NULL,
    sk_loja              INT          NOT NULL,
    sk_cargo            INT          NOT NULL,
    tipo_falta          VARCHAR(40)  NOT NULL,

```

```

        tipo_ocorrencia          VARCHAR(20)      NOT NULL,
        justificada              VARCHAR(3)       NOT NULL,
        cid                     VARCHAR(10)       NULL,
        total_dias_afastamento  INT              NOT NULL DEFAULT 0,
        horas_perdidas          DECIMAL(6,2)     NOT NULL DEFAULT 0.00,
        PRIMARY KEY (sk_absenteismo),
        KEY fk_fab_tempo_inicio (sk_tempo_inicio),
        KEY fk_fab_tempo_fim    (sk_tempo_fim),
        KEY fk_fab_colaborador  (sk_colaborador),
        KEY fk_fab_loja         (sk_loja),
        KEY fk_fab_cargo        (sk_cargo)
    );

-- =====
-- Popular fato_absenteismo
-- =====
INSERT INTO fato_absenteismo (
    sk_tempo_inicio,
    sk_tempo_fim,
    sk_colaborador,
    sk_loja,
    sk_cargo,
    tipo_falta,
    tipo_ocorrencia,
    justificada,
    cid,
    total_dias_afastamento,
    horas_perdidas
)
SELECT
    dt_ini.sk_tempo,
    dt_fim.sk_tempo,
    dc.sk_colaborador,
    dc.sk_loja,
    dc.sk_cargo,
    a.tipo_falta,
    a.tipo_ocorrencia,
    a.justificada,
    a.cid,
    a.total_dias_afastamento,
    a.horas_perdidas
FROM absenteismo a
INNER JOIN dim_tempo      dt_ini ON dt_ini.data_completa      = a.data_inicio_afastamento
INNER JOIN dim_tempo      dt_fim ON dt_fim.data_completa      = a.data_final_afastamento
INNER JOIN dim_colaborador dc     ON dc.matricula_colaborador = a.matricula_colaborador;

-- =====
-- Verificação
-- =====
SELECT
    COUNT(*)                AS total_ocorrencias,
    SUM(total_dias_afastamento) AS total_dias_afastados,
    ROUND(SUM(horas_perdidas), 2) AS total_horas_perdidas,
    ROUND(AVG(total_dias_afastamento), 2) AS media_dias_por_ocorrencia,
    COUNT(DISTINCT sk_colaborador) AS colaboradores_afetados
FROM fato_absenteismo;

```

fato_avaliacao_desempenho.sql

```

-- =====
-- FATO_AVALIACAO_DESEMPENHO
-- Granularidade: 1 linha por avaliação realizada
-- =====

DROP TABLE IF EXISTS fato_avaliacao_desempenho;

CREATE TABLE fato_avaliacao_desempenho (

```



```

sk_avaliacao          INT          NOT NULL AUTO_INCREMENT,
sk_tempo              INT          NOT NULL,
sk_colaborador        INT          NOT NULL,
sk_loja               INT          NOT NULL,
sk_cargo              INT          NOT NULL,
ano                   SMALLINT     NOT NULL,
ciclo_avaliacao       VARCHAR(10)  NOT NULL,
tipo_avaliacao        VARCHAR(20)  NOT NULL,
gestor_avaliador      VARCHAR(100) NOT NULL,
-- Notas por competência
comunicacao           DECIMAL(4,1)  NULL,
trabalho_em_equipe    DECIMAL(4,1)  NULL,
proatividade          DECIMAL(4,1)  NULL,
lideranca             DECIMAL(4,1)  NULL,
conhecimento_tecnico  DECIMAL(4,1)  NULL,
organizacao           DECIMAL(4,1)  NULL,
nota_final            DECIMAL(4,1)  NULL,
-- Classificação
classificacao_desempenho VARCHAR(15) NULL,
elegivel_promocao     VARCHAR(3)    NULL,
recomendacao_ajuste_salarial VARCHAR(3) NULL,
percentual_ajuste     DECIMAL(5,2)  NULL,
risco_desligamento   VARCHAR(6)    NULL,
PRIMARY KEY (sk_avaliacao),
KEY fk_fad_tempo      (sk_tempo),
KEY fk_fad_colaborador (sk_colaborador),
KEY fk_fad_loja       (sk_loja),
KEY fk_fad_cargo      (sk_cargo)
);

```

```

-- =====
-- Popular fato_avaliacao_desempenho
-- =====

```

```

INSERT INTO fato_avaliacao_desempenho (
    sk_tempo,
    sk_colaborador,
    sk_loja,
    sk_cargo,
    ano,
    ciclo_avaliacao,
    tipo_avaliacao,
    gestor_avaliador,
    comunicacao,
    trabalho_em_equipe,
    proatividade,
    lideranca,
    conhecimento_tecnico,
    organizacao,
    nota_final,
    classificacao_desempenho,
    elegivel_promocao,
    recomendacao_ajuste_salarial,
    percentual_ajuste,
    risco_desligamento
)

```

```

SELECT
    dt.sk_tempo,
    dc.sk_colaborador,
    dc.sk_loja,
    dc.sk_cargo,
    ad.ano,
    ad.ciclo_avaliacao,
    ad.tipo_avaliacao,
    ad.gestor_avaliador,
    ad.comunicacao,
    ad.trabalho_em_equipe,
    ad.proatividade,
    ad.lideranca,
    ad.conhecimento_tecnico,
    ad.organizacao,
    ad.nota_final,
    ad.classificacao_desempenho,

```

```

        ad.elegivel_promocao,
        ad.recomendacao_ajuste_salarial,
        ad.percentual_ajuste,
        ad.risco_desligamento
FROM avaliacao_desempenho ad
INNER JOIN dim_tempo      dt ON dt.data_completa      = ad.data_da_avaliacao
INNER JOIN dim_colaborador dc ON dc.matricula_colaborador = ad.matricula_colaborador;

-- =====
-- Verificação
-- =====
SELECT
    COUNT(*)                                AS total_avaliacoes,
    COUNT(DISTINCT sk_colaborador)           AS colaboradores_avaliados,
    ROUND(AVG(nota_final), 2)                AS media_nota_final,
    SUM(CASE WHEN classificacao_desempenho = 'Excelente'
            THEN 1 ELSE 0 END)               AS excelente,
    SUM(CASE WHEN classificacao_desempenho = 'Alto'
            THEN 1 ELSE 0 END)               AS alto,
    SUM(CASE WHEN classificacao_desempenho = 'Medio'
            THEN 1 ELSE 0 END)               AS medio,
    SUM(CASE WHEN classificacao_desempenho = 'Baixo'
            THEN 1 ELSE 0 END)

```

fato_headcount.sql

```

-- =====
-- PASSO 1: Recria fato_headcount
-- =====
DROP TABLE IF EXISTS fato_headcount;

CREATE TABLE fato_headcount (
    sk_headcount      INT          NOT NULL AUTO_INCREMENT,
    sk_tempo           INT          NOT NULL,
    sk_loja            INT          NOT NULL,
    sk_cargo           INT          NOT NULL,
    headcount_ativo    INT          NOT NULL DEFAULT 0,
    admissoes          INT          NOT NULL DEFAULT 0,
    demissoes          INT          NOT NULL DEFAULT 0,
    massa_salarial     DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    turnover_percentual DECIMAL(8,4) NOT NULL DEFAULT 0.00,
    headcount_orcado   INT          NULL,
    variacao_orcado    INT          NULL,
    PRIMARY KEY (sk_headcount),
    KEY fk_fhc_tempo (sk_tempo),
    KEY fk_fhc_loja  (sk_loja),
    KEY fk_fhc_cargo (sk_cargo)
);

-- =====
-- PASSO 2: Popula fato_headcount
-- =====
INSERT INTO fato_headcount (
    sk_tempo,
    sk_loja,
    sk_cargo,
    headcount_ativo,
    admissoes,
    demissoes,
    massa_salarial,
    turnover_percentual,
    headcount_orcado,
    variacao_orcado
)
SELECT
    a.sk_tempo,
    a.sk_loja,

```

```

a.sk_cargo,
a.headcount_ativo,
COALESCE(adm.total_admissoes, 0),
COALESCE(dem.total_demissoes, 0),
a.massa_salarial,
CASE
    WHEN a.headcount_ativo > 0
    THEN ROUND((COALESCE(dem.total_demissoes, 0) / a.headcount_ativo) * 100, 4)
    ELSE 0
END,
o.headcount_orcado,
CASE
    WHEN o.headcount_orcado IS NOT NULL
    THEN a.headcount_ativo - o.headcount_orcado
    ELSE NULL
END
FROM aux_ativos a
LEFT JOIN aux_admissoes adm ON adm.ano_mes = a.ano_mes
                        AND adm.sk_loja = a.sk_loja
                        AND adm.sk_cargo = a.sk_cargo
LEFT JOIN aux_demissoes dem ON dem.ano_mes = a.ano_mes
                        AND dem.sk_loja = a.sk_loja
                        AND dem.sk_cargo = a.sk_cargo
LEFT JOIN aux_orcado o ON o.ano_mes = a.ano_mes
                        AND o.sk_loja = a.sk_loja
                        AND o.sk_cargo = a.sk_cargo;

-- =====
-- PASSO 3: Verificação final
-- =====
SELECT
    COUNT(*)                AS total_linhas,
    SUM(headcount_ativo)    AS total_ativos,
    SUM(admissoes)          AS total_admissoes,
    SUM(demissoes)          AS total_demissoes,
    ROUND(AVG(turnover_percentual), 4) AS media_turnover,
    ROUND(SUM(massa_salarial), 2) AS massa_salarial_total
FROM fato_headcount;

-- =====
-- PASSO 4: Limpeza das tabelas auxiliares
-- =====
DROP TABLE IF EXISTS aux_meses;
DROP TABLE IF EXISTS aux_admissoes;
DROP TABLE IF EXISTS aux_demissoes;
DROP TABLE IF EXISTS aux_orcado;
DROP TABLE IF EXISTS aux_ativos;

```

fato_metas.sql

```

-- =====
-- FATO_METAS
-- Granularidade: 1 linha por centro de custo por mês
-- =====

DROP TABLE IF EXISTS fato_metas;

CREATE TABLE fato_metas (
    sk_meta          INT          NOT NULL AUTO_INCREMENT,
    sk_tempo         INT          NOT NULL,
    sk_loja          INT          NOT NULL,
    sk_cargo         INT          NULL,
    -- Metas
    turnover_percentual DECIMAL(5,2) NOT NULL DEFAULT 0.00,
    admissoes_quantidade INT       NOT NULL DEFAULT 0,
    demissoes_quantidade INT       NOT NULL DEFAULT 0,
    horas_extras     DECIMAL(8,2) NOT NULL DEFAULT 0.00,

```

```

absenteismo_percentual DECIMAL(5,2) NOT NULL DEFAULT 0.00,
treinamento_horas DECIMAL(8,2) NOT NULL DEFAULT 0.00,
-- Realizado (vindo das fatos)
admissoes_realizado INT NOT NULL DEFAULT 0,
demissoes_realizado INT NOT NULL DEFAULT 0,
turnover_realizado DECIMAL(8,4) NOT NULL DEFAULT 0.00,
horas_extras_realizado DECIMAL(10,2) NOT NULL DEFAULT 0.00,
treinamento_realizado DECIMAL(8,2) NOT NULL DEFAULT 0.00,
-- Variações meta x realizado
variacao_admissoes INT NOT NULL DEFAULT 0,
variacao_demissoes INT NOT NULL DEFAULT 0,
variacao_turnover DECIMAL(8,4) NOT NULL DEFAULT 0.00,
variacao_horas_extras DECIMAL(10,2) NOT NULL DEFAULT 0.00,
variacao_treinamento DECIMAL(8,2) NOT NULL DEFAULT 0.00,
PRIMARY KEY (sk_meta),
KEY fk_fmt_tempo (sk_tempo),
KEY fk_fmt_loja (sk_loja)
);

-- =====
-- Popular fato metas
-- =====
INSERT INTO fato metas (
    sk_tempo,
    sk_loja,
    turnover_percentual,
    admissoes_quantidade,
    demissoes_quantidade,
    horas_extras,
    absenteismo_percentual,
    treinamento_horas,
    admissoes_realizado,
    demissoes_realizado,
    turnover_realizado,
    horas_extras_realizado,
    treinamento_realizado,
    variacao_admissoes,
    variacao_demissoes,
    variacao_turnover,
    variacao_horas_extras,
    variacao_treinamento
)
SELECT
    dt.sk_tempo,
    dl.sk_loja,
    m.turnover_percentual,
    m.admissoes_quantidade,
    m.demissoes_quantidade,
    m.horas_extras,
    m.absenteismo_percentual,
    m.treinamento_horas,
    -- Realizado: admissões no mês
    COALESCE(hc.admissoes, 0),
    -- Realizado: demissões no mês
    COALESCE(hc.demissoes, 0),
    -- Realizado: turnover médio no mês
    COALESCE(hc.turnover_medio, 0),
    -- Realizado: horas extras no mês
    COALESCE(fp.horas_extras_realizado, 0),
    -- Realizado: horas treinamento no mês
    COALESCE(tr.treinamento_realizado, 0),
    -- Variações
    COALESCE(hc.admissoes, 0) - m.admissoes_quantidade,
    COALESCE(hc.demissoes, 0) - m.demissoes_quantidade,
    COALESCE(hc.turnover_medio, 0) - m.turnover_percentual,
    COALESCE(fp.horas_extras_realizado, 0) - m.horas_extras,
    COALESCE(tr.treinamento_realizado, 0) - m.treinamento_horas
FROM metas m
-- Join com dim_tempo pelo ano e mês
INNER JOIN dim_tempo dt ON dt.ano = m.ano
                        AND dt.mes = m.mes
                        AND dt.dia_mes = 1

```

```

-- Join com dim_loja pelo nome da loja
INNER JOIN dim_loja dl ON dl.nome_loja = m.nome_loja
-- Realizado headcount: soma admissões e demissões por loja no mês
LEFT JOIN (
    SELECT
        fh.sk_tempo,
        fh.sk_loja,
        SUM(fh.admissoes) AS admissoes,
        SUM(fh.demissoes) AS demissoes,
        ROUND(AVG(fh.turnover_percentual), 4) AS turnover_medio
    FROM fato_headcount fh
    GROUP BY fh.sk_tempo, fh.sk_loja
) hc ON hc.sk_tempo = dt.sk_tempo
    AND hc.sk_loja = dl.sk_loja
-- Realizado folha: soma horas extras por loja no mês
LEFT JOIN (
    SELECT
        ffp.sk_tempo,
        ffp.sk_loja,
        ROUND(SUM(ffp.horas_extras_valor), 2) AS horas_extras_realizado
    FROM fato_folha_pagamento ffp
    GROUP BY ffp.sk_tempo, ffp.sk_loja
) fp ON fp.sk_tempo = dt.sk_tempo
    AND fp.sk_loja = dl.sk_loja
-- Realizado treinamento: soma horas por loja no mês
LEFT JOIN (
    SELECT
        ftr.sk_tempo_inicio AS sk_tempo,
        ftr.sk_loja,
        SUM(ftr.horas_treinamento) AS treinamento_realizado
    FROM fato_treinamento ftr
    GROUP BY ftr.sk_tempo_inicio, ftr.sk_loja
) tr ON tr.sk_tempo = dt.sk_tempo
    AND tr.sk_loja = dl.sk_loja;

-- =====
-- Verificação final
-- =====
SELECT
    COUNT(*) AS total_linhas,
    ROUND(AVG(turnover_percentual), 2) AS media_turnover_meta,
    ROUND(AVG(turnover_realizado), 2) AS media_turnover_realizado,
    SUM(admissoes_quantidade) AS total_admissoes_meta,
    SUM(admissoes_realizado) AS total_admissoes_realizado,
    ROUND(SUM(horas_extras), 2) AS total_he_meta,
    ROUND(SUM(horas_extras_realizado), 2) AS total_he_realizado,
    ROUND(SUM(treinamento_horas), 2) AS total_trein_meta,
    ROUND(SUM(treinamento_realizado), 2) AS total_trein_realizado
FROM fato metas;

```

fato_recrutamento.sql

```

-- =====
-- Popular fato_recrutamento com joins corrigidos
-- =====
TRUNCATE TABLE fato_recrutamento;

INSERT INTO fato_recrutamento (
    sk_tempo_candidatura,
    sk_tempo_contratacao,
    sk_colaborador,
    sk_loja,
    sk_cargo,
    id_vaga,
    id_candidato,
    fonte_candidato,
    nivel_vaga,

```

```

        tipo_contratacao,
        status_candidato,
        aprovado,
        etapa_processo,
        tempo_contratacao_dias,
        tempo_por_etapa,
        salario_oferecido
    )
SELECT
    dt_cand.sk_tempo,
    dt_cont.sk_tempo,
    dc.sk_colaborador,
    dc.sk_loja,
    dc.sk_cargo,
    rs.id_vaga,
    rs.id_candidato,
    rs.fonte_candidato,
    rs.nivel_vaga,
    rs.tipo_contratacao,
    rs.status_candidato,
    rs.aprovado,
    rs.etapa_processo,
    rs.tempo_contratacao_dias,
    rs.tempo_por_etapa,
    rs.salario_oferecido
FROM recrutamento_selecao rs
INNER JOIN dim_tempo      dt_cand ON dt_cand.data_completa = rs.data_candidatura
INNER JOIN dim_tempo      dt_cont ON dt_cont.data_completa = rs.data_contratacao
INNER JOIN dim_colaborador dc      ON dc.nome_colaborador  = rs.nome_candidato
INNER JOIN dim_loja       dl      ON dl.cidade             = rs.cidade
INNER JOIN dim_cargo       dgc     ON dgc.funcao            = rs.funcao
                                   AND dgc.centro_custo      = rs.centro_custo
                                   AND dgc.departamento      = rs.departamento;

-- =====
-- Verificação
-- =====
SELECT
    COUNT(*)                                AS total_processos,
    COUNT(DISTINCT sk_colaborador)          AS colaboradores_contratados,
    ROUND(AVG(tempo_contratacao_dias), 2)   AS media_dias_contratacao,
    ROUND(AVG(salario_oferecido), 2)        AS media_salario_oferecido,
    SUM(CASE WHEN aprovado = 'Sim'
            THEN 1 ELSE 0 END)              AS total_aprovados,
    SUM(CASE WHEN nivel_vaga = 'Gestao'
            THEN 1 ELSE 0 END)              AS vagas_gestao,
    SUM(CASE WHEN nivel_vaga = 'Analista'
            THEN 1 ELSE 0 END)              AS vagas_analista,
    SUM(CASE WHEN nivel_vaga = 'Operacional'
            THEN 1 ELSE 0 END)              AS vagas_operacional
FROM fato_recrutamento;

```

fato_treinamentos.sql

```

-- =====
-- FATO_TREINAMENTO
-- Granularidade: 1 linha por treinamento realizado
-- =====

DROP TABLE IF EXISTS fato_treinamento;

CREATE TABLE fato_treinamento (
    sk_treinamento      INT          NOT NULL AUTO_INCREMENT,
    sk_tempo_inicio      INT          NOT NULL,
    sk_tempo_fim         INT          NOT NULL,
    sk_colaborador       INT          NOT NULL,
    sk_loja              INT          NOT NULL,

```

```

        sk_cargo          INT          NOT NULL,
        curso             VARCHAR(100) NOT NULL,
        horas_treinamento INT          NOT NULL DEFAULT 0,
        status            VARCHAR(20)  NOT NULL,
        PRIMARY KEY (sk_treinamento),
        KEY fk_ftr_tempo_inicio (sk_tempo_inicio),
        KEY fk_ftr_tempo_fim    (sk_tempo_fim),
        KEY fk_ftr_colaborador  (sk_colaborador),
        KEY fk_ftr_loja         (sk_loja),
        KEY fk_ftr_cargo        (sk_cargo)
    );

-- =====
-- Popular fato_treinamento
-- =====
INSERT INTO fato_treinamento (
    sk_tempo_inicio,
    sk_tempo_fim,
    sk_colaborador,
    sk_loja,
    sk_cargo,
    curso,
    horas_treinamento,
    status
)
SELECT
    dt_ini.sk_tempo,
    dt_fim.sk_tempo,
    dc.sk_colaborador,
    dc.sk_loja,
    dc.sk_cargo,
    t.curso,
    t.horas_treinamento,
    t.status
FROM treinamentos t
INNER JOIN dim_tempo      dt_ini ON dt_ini.data_completa = t.data_inicio
INNER JOIN dim_tempo      dt_fim ON dt_fim.data_completa = t.data_fim
INNER JOIN dim_colaborador dc     ON dc.matricula_colaborador = t.matricula_colaborador;

-- =====
-- Verificação
-- =====
SELECT
    COUNT(*)                AS total_treinamentos,
    SUM(horas_treinamento) AS total_horas,
    ROUND(AVG(horas_treinamento), 2) AS media_horas_por_treinamento,
    COUNT(DISTINCT sk_colaborador)    AS colaboradores_treinados,
    COUNT(DISTINCT curso)             AS cursos_distintos,
    SUM(CASE WHEN status = 'Concluído'
              THEN 1 ELSE 0 END)      AS concluidos,
    SUM(CASE WHEN status = 'Em Andamento'
              THEN 1 ELSE 0 END)      AS em_andamento
FROM fato_treinamento;

```

pop_aux.sql

```

-- =====
-- PASSO 5: Cria aux_ativos vazia primeiro
-- =====
DROP TABLE IF EXISTS aux_ativos;

CREATE TABLE aux_ativos (
    sk_tempo      INT          NOT NULL,
    ano_mes       VARCHAR(7)   NOT NULL,
    ano           INT          NOT NULL,
    mes           INT          NOT NULL,
    sk_loja       INT          NOT NULL,

```

```

        sk_cargo          INT          NOT NULL,
        headcount_ativo INT          NOT NULL,
        massa_salarial   DECIMAL(15,2) NOT NULL
    );

-- =====
-- PASSO 6: Insere ano a ano (execute um bloco por vez)
-- =====

-- ANO 2005
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2005
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2005 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2006
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2006
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2006 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2007
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2007
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2007 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2008
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2008
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2008 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2009
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2009
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;

```



```

SELECT 2009 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2010
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salarario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
       ON dc.data_admissao <= m.ultimo_dia
       AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2010
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2010 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2011
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salarario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
       ON dc.data_admissao <= m.ultimo_dia
       AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2011
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2011 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2012
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salarario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
       ON dc.data_admissao <= m.ultimo_dia
       AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2012
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2012 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2013
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salarario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
       ON dc.data_admissao <= m.ultimo_dia
       AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2013
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2013 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2014
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salarario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
       ON dc.data_admissao <= m.ultimo_dia
       AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2014
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2014 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2015
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salarario_contratacao)

```

```

FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2015
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2015 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2016
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2016
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2016 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2017
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2017
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2017 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2018
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2018
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2018 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2019
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2019
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2019 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2020
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
    ON dc.data_admissao <= m.ultimo_dia
    AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2020
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2020 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

```

```

-- ANO 2021
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
      ON dc.data_admissao <= m.ultimo_dia
      AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2021
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2021 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2022
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
      ON dc.data_admissao <= m.ultimo_dia
      AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2022
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2022 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2023
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
      ON dc.data_admissao <= m.ultimo_dia
      AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2023
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2023 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2024
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
      ON dc.data_admissao <= m.ultimo_dia
      AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2024
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2024 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2025
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc
      ON dc.data_admissao <= m.ultimo_dia
      AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2025
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2025 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- ANO 2026
INSERT INTO aux_ativos
SELECT m.sk_tempo, m.ano_mes, m.ano, m.mes,
       dc.sk_loja, dc.sk_cargo,
       COUNT(*), SUM(dc.salario_contratacao)
FROM aux_meses m
INNER JOIN dim_colaborador dc

```

```

        ON dc.data_admissao <= m.ultimo_dia
        AND (dc.data_demissao IS NULL OR dc.data_demissao > m.ultimo_dia)
WHERE m.ano = 2026
GROUP BY m.sk_tempo, m.ano_mes, m.ano, m.mes, dc.sk_loja, dc.sk_cargo;
SELECT 2026 AS ano_inserido, COUNT(*) AS linhas FROM aux_ativos;

-- =====
-- Verificação final
-- =====
SELECT COUNT(*) AS total_aux_ativos FROM aux_ativos;

```

pop_folha_pagamento_etapa.sql

```

-- POPULAR FOLHA DE PAGAMENTO POR ANO
INSERT INTO fato_folha_pagamento (
    sk_tempo, sk_colaborador, sk_loja, sk_cargo,
    salario_base, horas_extras_qtd, horas_extras_valor,
    adicional_noturno, bonus, comissoes,
    vale_alimentacao, vale_transporte, auxilio_combustivel, auxilio_creche,
    total_proventos, inss, irrf, total_descontos,
    salario_liquido, fgts, custo_total_empresa
)
SELECT
    dt.sk_tempo, dc.sk_colaborador, dc.sk_loja, dc.sk_cargo,
    fp.salario_base, fp.horas_extras_qtd, fp.horas_extras_valor,
    fp.adicional_noturno, fp.bonus, fp.comissoes,
    fp.vale_alimentacao, fp.vale_transporte, fp.auxilio_combustivel, fp.auxilio_creche,
    fp.total_proventos, fp.inss, fp.irrf, fp.total_descontos,
    fp.salario_liquido, fp.fgts, fp.custo_total_empresa
FROM folha_pagamento fp
INNER JOIN dim_tempo dt ON dt.ano_mes = fp.mes_ano AND dt.dia_mes = 1
INNER JOIN dim_colaborador dc ON dc.matricula_colaborador = fp.matricula_colaborador
WHERE fp.mes_ano LIKE '2026%';

-- Verifica acumulado
SELECT dt.ano, COUNT(*) AS linhas
FROM fato_folha_pagamento ffp
INNER JOIN dim_tempo dt ON dt.sk_tempo = ffp.sk_tempo
GROUP BY dt.ano ORDER BY dt.ano;

```

validacao_final.sql

```

-- =====
-- VALIDAÇÃO GERAL DE TODAS AS TABELAS
-- =====
SELECT 'dim_tempo' AS tabela, COUNT(*) AS total_linhas FROM dim_tempo
UNION ALL
SELECT 'dim_loja', COUNT(*) FROM dim_loja
UNION ALL
SELECT 'dim_cargo', COUNT(*) FROM dim_cargo
UNION ALL
SELECT 'dim_colaborador', COUNT(*) FROM dim_colaborador
UNION ALL
SELECT 'fato_headcount', COUNT(*) FROM fato_headcount
UNION ALL
SELECT 'fato_folha_pagamento', COUNT(*) FROM fato_folha_pagamento
UNION ALL
SELECT 'fato_absenteismo', COUNT(*) FROM fato_absenteismo
UNION ALL
SELECT 'fato_treinamento', COUNT(*) FROM fato_treinamento
UNION ALL
SELECT 'fato_avaliacao_desempenho', COUNT(*) FROM fato_avaliacao_desempenho
UNION ALL

```

```

SELECT 'fato_recrutamento',          COUNT(*) FROM fato_recrutamento
UNION ALL
SELECT 'fato_metas',                  COUNT(*) FROM fato_metas;

```

vw_absenteismo.sql

```

-- =====
-- VW_ABSENTEISMO_MENSAL
-- Absenteísmo por colaborador, loja e tipo de ocorrência
-- =====

DROP VIEW IF EXISTS vw_absenteismo_mensal;

CREATE VIEW vw_absenteismo_mensal AS
SELECT
  -- Tempo início
  dt_ini.ano,
  dt_ini.semestre,
  dt_ini.trimestre,
  dt_ini.mes,
  dt_ini.nome_mes,
  dt_ini.ano_mes,
  dt_ini.ano_trimestre,
  -- Loja
  dl.nome_loja,
  dl.tipo_loja,
  dl.cidade,
  dl.estado,
  dl.regiao,
  -- Cargo
  dc.departamento,
  dc.centro_custo,
  dc.funcao,
  dc.nivel_cargo,
  dc.familia_cargo,
  -- Colaborador
  col.matricula_colaborador,
  col.nome_colaborador,
  col.sexo,
  col.faixa_etaria,
  col.tipo_contratacao,
  -- Absenteísmo
  fab.tipo_falta,
  fab.tipo_ocorrencia,
  fab.justificada,
  fab.cid,
  fab.total_dias_afastamento,
  fab.horas_perdidas,
  -- Classificação
  CASE
    WHEN fab.total_dias_afastamento = 1 THEN 'Curta Duração'
    WHEN fab.total_dias_afastamento <= 7 THEN 'Média Duração'
    WHEN fab.total_dias_afastamento <= 30 THEN 'Longa Duração'
    ELSE 'Afastamento Prolongado'
  END AS classificacao_duracao,
  CASE
    WHEN fab.justificada = 'Sim' THEN 'Justificada'
    ELSE 'Injustificada'
  END AS status_justificativa
FROM fato_absenteismo fab
INNER JOIN dim_tempo      dt_ini ON dt_ini.sk_tempo      = fab.sk_tempo_inicio
INNER JOIN dim_tempo      dt_fim ON dt_fim.sk_tempo      = fab.sk_tempo_fim
INNER JOIN dim_loja        dl   ON dl.sk_loja            = fab.sk_loja
INNER JOIN dim_cargo       dc   ON dc.sk_cargo           = fab.sk_cargo
INNER JOIN dim_colaborador col   ON col.sk_colaborador    = fab.sk_colaborador;

-- =====

```

```

-- Verifica se foi criada
-- =====
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- =====
-- Verificação
-- =====
SELECT
    ano,
    tipo_ocorrencia,
    classificacao_duracao,
    COUNT(*) AS total_ocorrencias,
    SUM(total_dias_afastamento) AS total_dias,
    ROUND(SUM(horas_perdidas), 2) AS total_horas_perdidas,
    COUNT(DISTINCT matricula_colaborador) AS colaboradores_afetados
FROM vw_absenteismo_mensal
GROUP BY ano, tipo_ocorrencia, classificacao_duracao
ORDER BY ano, tipo_ocorrencia, classificacao_duracao;

```

vw_avaliacao.sql

```

-- =====
-- VW_AVALIACAO_DESEMPENHO
-- Histórico de avaliações com notas, classificações e riscos
-- =====

DROP VIEW IF EXISTS vw_avaliacao_desempenho;

CREATE VIEW vw_avaliacao_desempenho AS
SELECT
    -- Tempo
    dt.ano,
    dt.semestre,
    dt.trimestre,
    dt.mes,
    dt.nome_mes,
    dt.ano_mes,
    dt.ano_trimestre,
    -- Loja
    dl.nome_loja,
    dl.tipo_loja,
    dl.cidade,
    dl.estado,
    dl.regiao,
    -- Cargo
    dc.departamento,
    dc.centro_custo,
    dc.funcao,
    dc.nivel_cargo,
    dc.familia_cargo,
    -- Colaborador
    col.matricula_colaborador,
    col.nome_colaborador,
    col.sexo,
    col.faixa_etaria,
    col.escolaridade,
    col.tipo_contratacao,
    col.situacao,
    col.tempo_casa_anos,
    -- Avaliação
    fad.ano AS ano_avaliacao,
    fad.ciclo_avaliacao,
    fad.tipo_avaliacao,
    fad.gestor_avaliador,
    -- Notas por competência
    fad.comunicacao,
    fad.trabalho_em_equipe,

```

```

fad.proatividade,
fad.lideranca,
fad.conhecimento_tecnico,
fad.organizacao,
fad.nota_final,
-- Classificação e recomendações
fad.classificacao_desempenho,
fad.elegivel_promocao,
fad.recomendacao_ajuste_salarial,
fad.percentual_ajuste,
fad.risco_desligamento,
-- Classificação nota final
CASE
    WHEN fad.nota_final >= 9.0 THEN 'Excelente (9-10)'
    WHEN fad.nota_final >= 7.5 THEN 'Alto (7.5-8.9)'
    WHEN fad.nota_final >= 6.0 THEN 'Medio (6-7.4)'
    WHEN fad.nota_final >= 4.0 THEN 'Baixo (4-5.9)'
    ELSE 'Critico (0-3.9)'
END
-- Competência mais fraca
CASE
    WHEN LEAST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.comunicacao THEN 'Comunicacao'
    WHEN LEAST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.trabalho_em_equipe THEN 'Trabalho em Equipe'
    WHEN LEAST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.proatividade THEN 'Proatividade'
    WHEN LEAST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.conhecimento_tecnico THEN 'Conhecimento Tecnico'
    ELSE 'Organizacao'
END
-- Competência mais forte
CASE
    WHEN GREATEST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.comunicacao THEN 'Comunicacao'
    WHEN GREATEST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.trabalho_em_equipe THEN 'Trabalho em Equipe'
    WHEN GREATEST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.proatividade THEN 'Proatividade'
    WHEN GREATEST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.conhecimento_tecnico THEN 'Conhecimento Tecnico'
    ELSE 'Organizacao'
END
AS faixa_nota,

```

```

        fad.organizacao
    ) = fad.proatividade                                THEN 'Proatividade'
    WHEN GREATEST(
        fad.comunicacao,
        fad.trabalho_em_equipe,
        fad.proatividade,
        fad.conhecimento_tecnico,
        fad.organizacao
    ) = fad.conhecimento_tecnico                        THEN 'Conhecimento Tecnico'
    ELSE                                                'Organizacao'
    END                                                AS competencia_mais_forte
FROM fato_avaliacao_desempenho fad
INNER JOIN dim_tempo          dt ON dt.sk_tempo      = fad.sk_tempo
INNER JOIN dim_loja           dl ON dl.sk_loja        = fad.sk_loja
INNER JOIN dim_cargo          dc ON dc.sk_cargo       = fad.sk_cargo
INNER JOIN dim_colaborador    col ON col.sk_colaborador = fad.sk_colaborador;

-- =====
-- Verifica se foi criada
-- =====
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- =====
-- Verificação
-- =====
SELECT
    ano_avaliacao,
    departamento,
    classificacao_desempenho,
    COUNT(*)                                AS total_avaliacoes,
    ROUND(AVG(nota_final), 2)                AS media_nota,
    SUM(CASE WHEN elegivel_promocao = 'Sim'
        THEN 1 ELSE 0 END)                  AS elegiveis_promocao,
    SUM(CASE WHEN risco_desligamento = 'Alto'
        THEN 1 ELSE 0 END)                  AS risco_alto,
    SUM(CASE WHEN recomendacao_ajuste_salarial = 'Sim'
        THEN 1 ELSE 0 END)                  AS rec_ajuste_salarial
FROM vw_avaliacao_desempenho
GROUP BY ano_avaliacao, departamento, classificacao_desempenho
ORDER BY ano_avaliacao, departamento, classificacao_desempenho;

```

vw_folha.sql

```

-- =====
-- PASSO 1: Cria a view vw_folha_por_loja
-- =====
DROP VIEW IF EXISTS vw_folha_por_loja;

CREATE VIEW vw_folha_por_loja AS
SELECT
    dt.ano,
    dt.semestre,
    dt.trimestre,
    dt.mes,
    dt.nome_mes,
    dt.ano_mes,
    dt.ano_trimestre,
    dl.nome_loja,
    dl.tipo_loja,
    dl.cidade,
    dl.estado,
    dl.regiao,
    dc.departamento,
    dc.centro_custo,
    dc.funcao,
    dc.nivel_cargo,
    dc.familia_cargo,

```



```

col.tipo_contratacao,
col.sexo,
col.faixa_etaria,
col.escolaridade,
ffp.salario_base,
ffp.horas_extras_qtd,
ffp.horas_extras_valor,
ffp.adicional_noturno,
ffp.bonus,
ffp.comissoes,
ffp.vale_alimentacao,
ffp.vale_transporte,
ffp.auxilio_combustivel,
ffp.auxilio_creche,
ffp.total_proventos,
ffp.inss,
ffp.irrf,
ffp.total_descontos,
ffp.salario_liquido,
ffp.fgts,
ffp.custo_total_empresa
FROM fato_folha_pagamento ffp
INNER JOIN dim_tempo          dt  ON dt.sk_tempo          = ffp.sk_tempo
INNER JOIN dim_loja           dl  ON dl.sk_loja           = ffp.sk_loja
INNER JOIN dim_cargo           dc  ON dc.sk_cargo          = ffp.sk_cargo
INNER JOIN dim_colaborador    col ON col.sk_colaborador    = ffp.sk_colaborador;

-- =====
-- PASSO 2: Verifica se foi criada
-- =====
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- =====
-- PASSO 3: Verifica resultado
-- =====
SELECT
ano,
departamento,
ROUND(SUM(salario_base), 2)          AS total_salario_base,
ROUND(SUM(bonus), 2)                AS total_bonus,
ROUND(SUM(comissoes), 2)             AS total_comissoes,
ROUND(SUM(custo_total_empresa), 2)  AS custo_total_empresa,
ROUND(AVG(salario_base), 2)         AS media_salario
FROM vw_folha_por_loja
GROUP BY ano, departamento
ORDER BY ano, departamento;

```

vw_headcount.sql

```

-- =====
-- VW_HEADCOUNT_EVOLUCAO
-- Evolução mensal de headcount, admissões e demissões
-- por loja, departamento e cargo
-- =====

DROP VIEW IF EXISTS vw_headcount_evolucao;

CREATE VIEW vw_headcount_evolucao AS
SELECT
-- Tempo
dt.ano,
dt.semestre,
dt.trimestre,
dt.mes,
dt.nome_mes,
dt.ano_mes,
dt.ano_trimestre,

```

```

-- Loja
dl.nome_loja,
dl.tipo_loja,
dl.cidade,
dl.estado,
dl.regiao,
-- Cargo
dc.departamento,
dc.centro_custo,
dc.funcao,
dc.nivel_cargo,
dc.familia_cargo,
-- Métricas
fh.headcount_ativo,
fh.admissoes,
fh.demissoes,
fh.massa_salarial,
fh.headcount_orcado,
fh.variacao_orcado,
fh.turnover_percentual,
-- Métricas calculadas
ROUND(fh.massa_salarial / NULLIF(fh.headcount_ativo, 0), 2)
    AS salario_medio,
CASE
    WHEN fh.headcount_orcado IS NOT NULL AND fh.headcount_orcado > 0
    THEN ROUND((fh.headcount_ativo / fh.headcount_orcado) * 100, 2)
    ELSE NULL
END AS percentual_atendimento_orcado
FROM fato_headcount fh
INNER JOIN dim_tempo dt ON dt.sk_tempo = fh.sk_tempo
INNER JOIN dim_loja dl ON dl.sk_loja = fh.sk_loja
INNER JOIN dim_cargo dc ON dc.sk_cargo = fh.sk_cargo;

-- =====
-- Verificação
-- =====
SELECT
    ano,
    SUM(headcount_ativo)          AS total_ativos,
    SUM(admissoes)                AS total_admissoes,
    SUM(demissoes)               AS total_demissoes,
    ROUND(AVG(turnover_percentual), 4) AS media_turnover
FROM vw_headcount_evolucao
GROUP BY ano
ORDER BY ano;

```

vw_metas.sql

```

-- =====
-- VW_METAS_VS_REALIZADO
-- Comparativo completo entre metas e realizado por loja/mês
-- =====

DROP VIEW IF EXISTS vw_metas_vs_realizado;

CREATE VIEW vw_metas_vs_realizado AS
SELECT
    -- Tempo
    dt.ano,
    dt.semestre,
    dt.trimestre,
    dt.mes,
    dt.nome_mes,
    dt.ano_mes,
    dt.ano_trimestre,
    -- Loja
    dl.nome_loja,

```

```

dl.tipo_loja,
dl.cidade,
dl.estado,
dl.regiao,
-- Metas
fm.turnover_percentual          AS meta_turnover,
fm.admissoes_quantidade         AS meta_admissoes,
fm.demissoes_quantidade        AS meta_demissoes,
fm.horas_extras                AS meta_horas_extras,
fm.absenteismo_percentual       AS meta_absenteismo,
fm.treinamento_horas          AS meta_treinamento_horas,
-- Realizado
fm.admissoes_realizado          AS realizado_admissoes,
fm.demissoes_realizado         AS realizado_demissoes,
fm.turnover_realizado          AS realizado_turnover,
fm.horas_extras_realizado      AS realizado_horas_extras,
fm.treinamento_realizado      AS realizado_treinamento_horas,
-- Variações meta x realizado
fm.variacao_admissoes,
fm.variacao_demissoes,
fm.variacao_turnover,
fm.variacao_horas_extras,
fm.variacao_treinamento,
-- Status admissões
CASE
    WHEN fm.admissoes_realizado >= fm.admissoes_quantidade THEN 'Atingido'
    WHEN fm.admissoes_realizado >= fm.admissoes_quantidade
        * 0.9 THEN 'Quase Atingido'
    ELSE 'Nao Atingido'
END AS status_admissoes,
-- Status demissões
CASE
    WHEN fm.demissoes_realizado <= fm.demissoes_quantidade THEN 'Atingido'
    WHEN fm.demissoes_realizado <= fm.demissoes_quantidade
        * 1.1 THEN 'Quase Atingido'
    ELSE 'Nao Atingido'
END AS status_demissoes,
-- Status turnover
CASE
    WHEN fm.turnover_realizado <= fm.turnover_percentual THEN 'Atingido'
    WHEN fm.turnover_realizado <= fm.turnover_percentual
        * 1.1 THEN 'Quase Atingido'
    ELSE 'Nao Atingido'
END AS status_turnover,
-- Status horas extras
CASE
    WHEN fm.horas_extras_realizado <= fm.horas_extras THEN 'Atingido'
    WHEN fm.horas_extras_realizado <= fm.horas_extras
        * 1.1 THEN 'Quase Atingido'
    ELSE 'Nao Atingido'
END AS status_horas_extras,
-- Status treinamento
CASE
    WHEN fm.treinamento_realizado >= fm.treinamento_horas THEN 'Atingido'
    WHEN fm.treinamento_realizado >= fm.treinamento_horas
        * 0.9 THEN 'Quase Atingido'
    ELSE 'Nao Atingido'
END AS status_treinamento,
-- Score geral de metas (quantas metas foram atingidas)
(
    CASE WHEN fm.admissoes_realizado >= fm.admissoes_quantidade
        THEN 1 ELSE 0 END +
    CASE WHEN fm.demissoes_realizado <= fm.demissoes_quantidade
        THEN 1 ELSE 0 END +
    CASE WHEN fm.turnover_realizado <= fm.turnover_percentual
        THEN 1 ELSE 0 END +
    CASE WHEN fm.horas_extras_realizado <= fm.horas_extras
        THEN 1 ELSE 0 END +
    CASE WHEN fm.treinamento_realizado >= fm.treinamento_horas
        THEN 1 ELSE 0 END
) AS metas_atingidas,
-- Classificação geral

```

```

CASE
    WHEN (
        CASE WHEN fm.admissoes_realizado >= fm.admissoes_quantidade
            THEN 1 ELSE 0 END +
        CASE WHEN fm.demissoes_realizado <= fm.demissoes_quantidade
            THEN 1 ELSE 0 END +
        CASE WHEN fm.turnover_realizado <= fm.turnover_percentual
            THEN 1 ELSE 0 END +
        CASE WHEN fm.horas_extras_realizado <= fm.horas_extras
            THEN 1 ELSE 0 END +
        CASE WHEN fm.treinamento_realizado >= fm.treinamento_horas
            THEN 1 ELSE 0 END
    ) = 5
    THEN 'Excelente'
    WHEN (
        CASE WHEN fm.admissoes_realizado >= fm.admissoes_quantidade
            THEN 1 ELSE 0 END +
        CASE WHEN fm.demissoes_realizado <= fm.demissoes_quantidade
            THEN 1 ELSE 0 END +
        CASE WHEN fm.turnover_realizado <= fm.turnover_percentual
            THEN 1 ELSE 0 END +
        CASE WHEN fm.horas_extras_realizado <= fm.horas_extras
            THEN 1 ELSE 0 END +
        CASE WHEN fm.treinamento_realizado >= fm.treinamento_horas
            THEN 1 ELSE 0 END
    ) >= 3
    THEN 'Bom'
    WHEN (
        CASE WHEN fm.admissoes_realizado >= fm.admissoes_quantidade
            THEN 1 ELSE 0 END +
        CASE WHEN fm.demissoes_realizado <= fm.demissoes_quantidade
            THEN 1 ELSE 0 END +
        CASE WHEN fm.turnover_realizado <= fm.turnover_percentual
            THEN 1 ELSE 0 END +
        CASE WHEN fm.horas_extras_realizado <= fm.horas_extras
            THEN 1 ELSE 0 END +
        CASE WHEN fm.treinamento_realizado >= fm.treinamento_horas
            THEN 1 ELSE 0 END
    ) >= 2
    THEN 'Regular'
    ELSE
        'Critico'
    END
    AS classificacao_geral
FROM fato_metas fm
INNER JOIN dim_tempo dt ON dt.sk_tempo = fm.sk_tempo
INNER JOIN dim_loja dl ON dl.sk_loja = fm.sk_loja;

-- =====
-- Verifica se foi criada
-- =====
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- =====
-- Verificação final
-- =====
SELECT
    ano,
    classificacao_geral,
    COUNT(*) AS total_registros,
    ROUND(AVG(meta_turnover), 2) AS media_meta_turnover,
    ROUND(AVG(realizado_turnover), 4) AS media_realizado_turnover,
    SUM(CASE WHEN status_admissoes = 'Atingido'
        THEN 1 ELSE 0 END) AS admissoes_atingidas,
    SUM(CASE WHEN status_treinamento = 'Atingido'
        THEN 1 ELSE 0 END) AS treinamento_atingido,
    ROUND(AVG(metas_atingidas), 2) AS media_metas_atingidas
FROM vw_metas_vs_realizado
GROUP BY ano, classificacao_geral
ORDER BY ano, classificacao_geral;

```

vw_recrutamento_funil.sql

```

-- =====
-- VW_RECRUTAMENTO_FUNIL
-- Funil de recrutamento com tempo médio por etapa
-- =====

DROP VIEW IF EXISTS vw_recrutamento_funil;

CREATE VIEW vw_recrutamento_funil AS
SELECT
    -- Tempo candidatura
    dt_cand.ano,
    dt_cand.semestre,
    dt_cand.trimestre,
    dt_cand.mes,
    dt_cand.nome_mes,
    dt_cand.ano_mes,
    dt_cand.ano_trimestre,
    -- Loja
    dl.nome_loja,
    dl.tipo_loja,
    dl.cidade,
    dl.estado,
    dl.regiao,
    -- Cargo
    dc.departamento,
    dc.centro_custo,
    dc.funcao,
    dc.nivel_cargo,
    dc.familia_cargo,
    -- Colaborador contratado
    col.matricula_colaborador,
    col.nome_colaborador,
    col.sexo,
    col.faixa_etaria,
    col.escolaridade,
    -- Recrutamento
    fr.id_vaga,
    fr.id_candidato,
    fr.fonte_candidato,
    fr.nivel_vaga,
    fr.tipo_contratacao,
    fr.status_candidato,
    fr.aprovado,
    fr.etapa_processo,
    fr.tempo_contratacao_dias,
    fr.tempo_por_etapa,
    fr.salario_oferecido,
    -- Classificação tempo de contratação
    CASE
        WHEN fr.tempo_contratacao_dias <= 15 THEN 'Rápido (até 15 dias)'
        WHEN fr.tempo_contratacao_dias <= 30 THEN 'Normal (16 a 30 dias)'
        WHEN fr.tempo_contratacao_dias <= 45 THEN 'Lento (31 a 45 dias)'
        ELSE 'Muito Lento (mais de 45 dias)'
    END AS classificacao_tempo,
    -- Classificação salário oferecido
    CASE
        WHEN fr.salario_oferecido <= 2000 THEN 'Até R$ 2.000'
        WHEN fr.salario_oferecido <= 5000 THEN 'R$ 2.001 a R$ 5.000'
        WHEN fr.salario_oferecido <= 10000 THEN 'R$ 5.001 a R$ 10.000'
        WHEN fr.salario_oferecido <= 20000 THEN 'R$ 10.001 a R$ 20.000'
        ELSE 'Acima de R$ 20.000'
    END AS faixa_salarial_oferecida,
    -- Ano de contratação
    dt_cont.ano AS ano_contratacao,
    dt_cont.ano_mes AS ano_mes_contratacao
FROM fato_recrutamento fr
INNER JOIN dim_tempo dt_cand ON dt_cand.sk_tempo = fr.sk_tempo_candidatura
INNER JOIN dim_tempo dt_cont ON dt_cont.sk_tempo = fr.sk_tempo_contratacao
INNER JOIN dim_loja dl ON dl.sk_loja = fr.sk_loja
INNER JOIN dim_cargo dc ON dc.sk_cargo = fr.sk_cargo
INNER JOIN dim_colaborador col ON col.sk_colaborador = fr.sk_colaborador;

```

```

-- =====
-- Verifica se foi criada
-- =====
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- =====
-- Verificação
-- =====
SELECT
    ano,
    departamento,
    nivel_vaga,
    fonte_candidato,
    classificacao_tempo,
    COUNT(*) AS total_contratacoes,
    ROUND(AVG(tempo_contratacao_dias), 2) AS media_dias_contratacao,
    ROUND(AVG(salario_oferecido), 2) AS media_salario_oferecido,
    SUM(CASE WHEN aprovado = 'Sim'
           THEN 1 ELSE 0 END) AS aprovados
FROM vw_recrutamento_funil
GROUP BY
    ano,
    departamento,
    nivel_vaga,
    fonte_candidato,
    classificacao_tempo
ORDER BY ano, departamento, nivel_vaga;

```

vw_treinamento.sql

```

-- =====
-- VW_TREINAMENTO_POR_COLABORADOR
-- Histórico de treinamentos por colaborador, loja e cargo
-- =====

DROP VIEW IF EXISTS vw_treinamento_por_colaborador;

CREATE VIEW vw_treinamento_por_colaborador AS
SELECT
    -- Tempo
    dt_ini.ano,
    dt_ini.semestre,
    dt_ini.trimestre,
    dt_ini.mes,
    dt_ini.nome_mes,
    dt_ini.ano_mes,
    dt_ini.ano_trimestre,
    -- Loja
    dl.nome_loja,
    dl.tipo_loja,
    dl.cidade,
    dl.estado,
    dl.regiao,
    -- Cargo
    dc.departamento,
    dc.centro_custo,
    dc.funcao,
    dc.nivel_cargo,
    dc.familia_cargo,
    -- Colaborador
    col.matricula_colaborador,
    col.nome_colaborador,
    col.sexo,
    col.faixa_etaria,
    col.escolaridade,
    col.tipo_contratacao,
    col.situacao,

```

```

-- Treinamento
ftr.curso,
ftr.horas_treinamento,
ftr.status,
-- Classificação carga horária
CASE
    WHEN ftr.horas_treinamento <= 8 THEN 'Curto (até 8h)'
    WHEN ftr.horas_treinamento <= 24 THEN 'Médio (9h a 24h)'
    WHEN ftr.horas_treinamento <= 40 THEN 'Longo (25h a 40h)'
    ELSE 'Extenso (mais de 40h)'
END AS classificacao_carga,
-- Dias de duração do treinamento
dt_fim.data_completa - dt_ini.data_completa + 1 AS dias_duracao
FROM fato_treinamento ftr
INNER JOIN dim_tempo dt_ini ON dt_ini.sk_tempo = ftr.sk_tempo_inicio
INNER JOIN dim_tempo dt_fim ON dt_fim.sk_tempo = ftr.sk_tempo_fim
INNER JOIN dim_loja dl ON dl.sk_loja = ftr.sk_loja
INNER JOIN dim_cargo dc ON dc.sk_cargo = ftr.sk_cargo
INNER JOIN dim_colaborador col ON col.sk_colaborador = ftr.sk_colaborador;

-- =====
-- Verifica se foi criada
-- =====
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- =====
-- Verificação
-- =====
SELECT
    ano,
    departamento,
    status,
    classificacao_carga,
    COUNT(*) AS total_treinamentos,
    SUM(horas_treinamento) AS total_horas,
    ROUND(AVG(horas_treinamento), 2) AS media_horas,
    COUNT(DISTINCT matricula_colaborador) AS colaboradores_treinados
FROM vw_treinamento_por_colaborador
GROUP BY ano, departamento, status, classificacao_carga
ORDER BY ano, departamento, status, classificacao_carga;

```

vw_turnover.sql

```

-- =====
-- Vw_TURNOVER_MENSAL – versão ajustada com granularidade correta
-- =====

DROP VIEW IF EXISTS vw_turnover_mensal;

CREATE VIEW vw_turnover_mensal AS

-- 1. AGREGA FATO (remove granularidade de cargo)
WITH base_headcount AS (
    SELECT
        sk_tempo,
        sk_loja,
        SUM(headcount_ativo) AS headcount_ativo,
        SUM(admissoes) AS admissoes,
        SUM(demissoes) AS demissoes,
        ROUND(AVG(turnover_percentual), 4) AS turnover_percentual
    FROM fato_headcount
    GROUP BY sk_tempo, sk_loja
),

-- 2. AGREGA METAS (garante consistência)
base_metas AS (
    SELECT

```

```

        sk_tempo,
        sk_loja,
        ROUND(AVG(turnover_percentual), 4) AS turnover_meta
    FROM fato metas
    GROUP BY sk_tempo, sk_loja
)

SELECT
    -- Tempo
    dt.ano,
    dt.trimestre,
    dt.mes,
    dt.nome_mes,
    dt.ano_mes,
    dt.ano_trimestre,

    -- Loja
    dl.nome_loja,
    dl.tipo_loja,
    dl.cidade,
    dl.estado,
    dl.regiao,

    -- Métricas
    bh.headcount_ativo,
    bh.admissoes,
    bh.demissoes,
    bh.turnover_percentual                AS turnover_realizado,

    bm.turnover_meta,

    -- Variação
    ROUND(bh.turnover_percentual - COALESCE(bm.turnover_meta, 0), 4)
                                                AS variacao_turnover,

    -- Classificação
    CASE
        WHEN bh.turnover_percentual = 0 THEN 'Sem Movimentacao'
        WHEN bh.turnover_percentual <= 1.5 THEN 'Saudavel'
        WHEN bh.turnover_percentual <= 3.0 THEN 'Atencao'
        WHEN bh.turnover_percentual <= 5.0 THEN 'Critico'
        ELSE 'Muito Critico'
    END
                                                AS classificacao_turnover,

    -- Status meta
    CASE
        WHEN bm.turnover_meta IS NULL THEN 'Sem Meta'
        WHEN bh.turnover_percentual <= bm.turnover_meta THEN 'Dentro da Meta'
        ELSE 'Acima da Meta'
    END
                                                AS status_meta

FROM base_headcount bh
INNER JOIN dim_tempo dt ON dt.sk_tempo = bh.sk_tempo
INNER JOIN dim_loja dl ON dl.sk_loja = bh.sk_loja
LEFT JOIN base metas bm
    ON bm.sk_tempo = bh.sk_tempo
    AND bm.sk_loja = bh.sk_loja;

```